

Witnessed Independence: Recording Whether a Verifier Graded Its Own Work

Zain Dana Harper

Seattle, WA | [ORCID 0009-0001-7175-5393](https://orcid.org/0009-0001-7175-5393)

<https://github.com/HarperZ9>

July 2026

Abstract

A verifier is a program that decides whether some object is correct. Its decision is only worth trusting when the check is independent of the thing it judges. A check written by the same party that produced the object can pass and still mean nothing. In practice this independence is usually an unrecorded assumption of coding discipline. The reviewer trusts that the judge and the producer are different parties, and nothing in the verdict says whether they are. We describe a small, shipped mechanism that turns this assumption into recorded state that can be re-checked. A verification operation carries an optional provenance identity, a recorded name for who made something, in this case the party that authored the judging criterion, and a second optional provenance identity for the party that produced the object. From the pair it resolves a three-valued independence annotation: WITNESSED-INDEPENDENT, SELF-AUTHORED, or UNWITNESSED. At an opt-in trust boundary it downgrades a decided verdict to UNVERIFIABLE when independence is not positively witnessed. Absence of a witness is reported as UNWITNESSED and is never assumed independent. The mechanism can therefore label a genuinely independent check UNWITNESSED when the identities were not supplied, and it never reports independence that was not witnessed. The default mode leaves an existing call's verdict content byte-for-byte unchanged and adds only the honest label UNWITNESSED, so the annotation is additive. We place the mechanism inside a single verification operation that perceives an object, judges it against a criterion it did not author, and carries a certificate that can be re-checked. We give two instantiated criteria that exercise the same operation: grounded creative novelty against a witnessed corpus, and composition-sound provenance. This is a tested library primitive. The independence mechanism and its criteria pass a named unit suite. It is not a validated deployment across a fleet of tools. We state adoption beyond the components shown as future work and do not claim it.

In plain terms

Think about a school exam. A student writes answers on a test. Later, someone grades the test against an answer key. The grade is the verdict. The answer key is the criterion, the rule used to judge. We call the student's answers the artifact, the thing being judged.

A grade means something only when the person who grades is not the same person who took the test. Imagine the student writes the answers, then writes the answer key, then grades their own test. A pass from that arrangement tells you nothing. The student could have written a key that matches whatever they wrote. Now imagine a different teacher grades the same test against a key that teacher prepared. The same pass now carries weight, because the check and the work came from different people.

This paper fixes a specific problem. In many systems the grade is recorded, but nothing next to the grade says which of the two arrangements produced it. A self-graded pass looks

exactly like an independently graded pass. The intention, to grade work against a rule you did not write yourself, was real, and nothing let you see whether it held.

The fix is to record, right next to the verdict, who wrote the rule and who made the work. If both names are present and different, the verdict is labeled `WITNESSED-INDEPENDENT`. If both names are present and the same, it is labeled `SELF-AUTHORED`. If either name is missing, it is labeled `UNWITNESSED`, which is the honest way of saying we do not know. Missing information is never upgraded to independent. That would be a guess, and a guess here is the failure we are trying to avoid.

Recording the label is the default, and it changes nothing else. Two verdicts that differ only in a missing name stay byte-for-byte the same as before, plus the label `UNWITNESSED`. On top of that, a system can turn on a stricter rule at a chosen boundary. In the strict rule, a decided pass or fail from a self-graded check is turned into `UNVERIFIABLE`, which means the system refuses to treat a self-graded decision as a checked one. In the stronger rule, called `require-independent`, a decided verdict is turned into `UNVERIFIABLE` unless the label is `WITNESSED-INDEPENDENT`, so both the self-graded case and the unknown case are refused. These rules only withhold a decision. They never create a pass that was not already there.

The paper shows this working in a real, small piece of software, with two example checks built on the same operation. One check asks whether a new picture is different from every picture in a folder of earlier work, or a near-copy of one of them. The other check asks whether a file’s origin labels agree with each other. A file can carry one label saying a human made it and another saying a machine made it, and the check catches that disagreement. Both reduce to the same act: look at the object, judge it by a rule you did not write, and keep a record that someone else can re-check.

1 Introduction

Plainly: automated checks usually fail because the check is not independent of the work it judges, even when the check is correct, and until now nothing in the verdict recorded whether independence held.

The recurring failure of automated verification is a check that is not *independent* of the work it judges. The check can be correct and still fail this way. When the party that produces an object also writes the check that clears it, the verdict is a tautology dressed as evidence. A model that answers can grade its own answer. The same pattern appears when a pipeline attests to the result it emitted, or when an agent certifies the action it took. The machine-learning literature has met this directly in the setting where a language model is used as a judge. There a model that evaluates its own output, or a sibling model’s output, shows self-preference and can be pushed into approving what it should reject [4]. The classical response is proof-carrying code [1], and more broadly, verdicts that travel with evidence that can be re-derived. With re-derivable evidence, a reviewer can re-run the check and confirm it, so the producer’s word is no longer the basis of trust. But re-derivability alone does not establish independence. A producer can ship a check that re-derives perfectly and that the producer also wrote. A re-runner will then faithfully reproduce a pass that the producer graded for itself.

The gap we address is narrow and concrete. In a verification system we built, the design principle was stated plainly: judge an object against a criterion it did not author. Yet nothing in a verdict recorded whether that principle held. A criterion carried no record of its author, and the operation never recorded who produced the object. A self-graded verdict was therefore byte-indistinguishable from an independent one. The system intended independence, and no record showed whether any given verdict achieved it.

This paper describes the mechanism that closes that specific gap, and only that gap. It records, inside the verdict itself, a witness of whether the judging criterion and the object were authored by the same party. At an explicit trust boundary it also refuses to turn a self-

graded or unwitnessed decision into a positive verdict. The mechanism is not a new notion of independence, a new cryptographic primitive, or a learned evaluator.

Contributions.

1. **Witnessed independence** (Section 3): a three-valued annotation that preserves the default. It makes “a criterion it did not author” a recorded field of a verdict. Before, this was an unstated assumption. It has two opt-in downgrade modes and a stated bias toward soundness.
2. **The host operation** (Section 2): the annotation is a property of one verification operation that perceives, judges, and witnesses. The operation is fail-closed, so any failure in perception or judging degrades to UNVERIFIABLE. In default mode it is verdict-faithful, so the criterion’s own verdict is passed through unchanged. The annotation therefore composes without changing existing behavior.
3. **Two exercised criteria** (Section 4): grounded creative novelty against a witnessed corpus, which is a check against mode collapse, and composition-sound provenance, which resolves authenticity signals that contradict each other. Each is a total function, and each reduces to the same operation.
4. **An honest boundary** (Section 5): a named unit suite substantiates the mechanism and its criteria. Fleet-wide adoption is explicitly future work, and we identify what would falsify the broader claim.

2 The host operation

Plainly: the label lives inside one small operation that already fails safe and never changes a verdict, so adding the label changes nothing that was already working.

The mechanism lives inside a single operation we will call *reconcile*. Given an object, the operation does three things. It *perceives* the object into a witnessed form, together with the bytes that form was derived from. A criterion then *judges* that form. The operation returns one *observation* carrying the criterion’s verdict, its cited evidence, a content hash of the claim, and a witness of the payload bytes. Two properties make it a safe host for an added annotation. First, it is fail-closed. Any exception in perception, judging, or result construction, including a malformed certificate from an untrusted criterion, collapses to an UNVERIFIABLE observation and does not propagate. Second, in its default mode it is verdict-faithful. It never alters the criterion’s verdict. The recorded verdict is the criterion’s own, passed through. The opt-in guards of Section 3 are the sole, deliberate exception. They only ever downgrade a decided verdict toward UNVERIFIABLE, and never manufacture one. The operation therefore adds accountability structure around a verdict without ever inventing one.

3 Witnessed independence

Plainly: from the two recorded names the system picks one of three honest labels. By default it keeps the old behavior. When you ask it to be strict, it refuses to decide unless independence is witnessed.

The judging criterion optionally carries an **author** identity, the party that wrote the judge. The operation separately accepts an optional **producer** identity for the party that produced the object. From the pair, a pure resolver assigns one of three values:

- WITNESSED-INDEPENDENT: both present *and* different.
- SELF-AUTHORED: both present *and* equal.

- UNWITNESSED: either absent. This is the default.

Three design commitments follow.

Default-preserving. When independence is UNWITNESSED, the author and producer identities are not recorded at all. Two verdicts that differ only in an unwitnessed identity are byte-identical, exactly as they were before the mechanism existed. The annotation is strictly additive. An existing call site is unchanged in content and only gains the honest label UNWITNESSED.

Bias toward soundness. The absence of a witness is recorded as UNWITNESSED, and the mechanism never records it as WITNESSED-INDEPENDENT. An unsupplied witness is honestly unknown. It is never assumed to establish independence. The mechanism therefore never reports independence that was not witnessed. When identities are missing, it can fail to report independence that in fact held.

Opt-in refusal at a trust boundary. Two modes, both off by default, convert the annotation into an enforced guard on a *decided* verdict, which is a positive or negative decision, distinct from UNVERIFIABLE:

- *strict*: a decided verdict from a SELF-AUTHORED criterion is downgraded to UNVERIFIABLE with a stable reason. The system refuses to launder a self-graded decision as a witnessed one.
- *require-independent*: the stronger guard. A decided verdict is downgraded unless independence is WITNESSED-INDEPENDENT, so *both* self-authored and unwitnessed decisions are refused. This encodes the principle that the mere absence of an external check must not be read as truth.

Because both modes only ever downgrade toward UNVERIFIABLE, they cannot manufacture a positive verdict. The worst they do is refuse to decide. The downgrade records its reason, so a reader can see why a decision was withheld.

4 Two exercised criteria

Plainly: the same operation runs two real checks, one for whether a picture is a copy and one for whether a file’s origin labels contradict each other.

The operation is generic over the criterion, so independent domains reduce to it by supplying a judge. We give two that were built as motivating instances.

Grounded creative novelty. A reported failure mode of naive loops that generate a work, critique it, and regenerate it is collapse to a handful of generic motifs when the critic is an ungrounded caption [5]. A grounded criterion judges a work’s perceived signature against a *witnessed corpus* of prior work. It returns VERIFIED-novel only if the minimum distance to every corpus signature meets a threshold, REFUTED-derivative if the work collapses onto something already seen, and UNVERIFIABLE on an empty corpus. The criterion is generic over the signature type and distance metric. A perceptual hash with Hamming distance is the shipped instantiation. This establishes non-derivativeness *in the perceived signature*. It does not establish deep semantic novelty.

Composition-sound provenance. Uncoordinated authenticity layers can each pass even when they jointly contradict one another. A file can carry a valid “human-authored” manifest and an “AI-generated” watermark at the same time, because the layers never consult one another [6]. The fix composes the layers: map each provenance signal onto a verdict lattice and take the proven meet, so any denying or contradicting signal refutes the origin, all-affirming signals verify it, and absence attenuates to UNVERIFIABLE. The certificate records each signal’s raw value and its mapped verdict. The dominating signal, which is the answer to “which one refuted origin?”, is recoverable from the proof.

5 Evaluation and honest scope

Plainly: the code and its checks pass a named test suite. This is one small building block. It is not a whole system, and we say exactly what would prove the bigger claim wrong.

What is substantiated. The mechanism and its criteria are shipped, tested code. The reconcile operation, the independence resolver, and the two downgrade modes are covered by a named unit suite. Among its cases is a demonstration that a pre-existing shipped verifier organ yields a byte-identical verdict and oracle when re-expressed as this operation under the same criterion. Two further cases assert that *require-independent* downgrades both an UNWITNESSED and a SELF-AUTHORED decided verdict to UNVERIFIABLE, while recording the correct independence value. The two criteria pass their own cases, including the anti-collapse guard, in which a degenerate zero threshold is rejected at construction, and the contradiction case, in which a denying signal refutes a composed origin.

What is not. This is a library primitive. It is not a validated deployment. The operation is imported so far by the verifier’s own organs, its accountable-memory re-verification path, one external provenance-composition tool, and internal specification-verification scripts. It is *not* imported by the named flagship fleet the surrounding project describes, and we make no claim that “every verification surface is this operation.” That broader claim is future work, and it is falsifiable. It fails the moment a verification surface in the fleet cannot be expressed as this operation without changing its verdicts. The mechanism has limits. It witnesses *authorship* independence through declared identities, so it is only as meaningful as those identities are honest. It does not establish semantic correctness. And colluding but distinct identities would read as WITNESSED-INDEPENDENT.

6 Related work

Plainly: earlier work moves trust onto a re-checkable artifact or onto types, but none of it records who wrote the check, and that is what this adds.

Proof-carrying code [1] and proof-carrying results generally move trust from the producer to a re-checkable artifact. Independence-witnessing is complementary. It records *who* authored the check that is being re-derived. Refinement and dependent types [2] enforce properties at the type level, but they do not annotate the provenance of the property’s author. Content-provenance standards such as C2PA [3] carry authorship claims, but, as the authenticated-contradictions result shows [6], they do not by themselves compose across layers. Our origin criterion is one composition rule over such signals. The LLM-as-judge literature [4] documents the self-grading pathology empirically. Our contribution is a recorded, enforceable statement of whether a judge was independent at all. It is not a better judge.

7 Conclusion

Plainly: independence is what makes a verdict mean something, and this is a small, careful way to record it and to refuse to fake it.

Independence is the property that makes a verdict mean something, and it is usually unrecorded. We described a small mechanism that records it: a three-valued witness, preserving the default, of whether the judging criterion and the object share an author, together with an opt-in refusal to decide when independence is not positively witnessed. The design stays conservative and additive, and it states its own reach honestly. The primitive is tested. Its fleet-wide generalization remains a falsifiable open question, and we do not present it as a result.

References

- [1] Necula, G. C. *Proof-carrying code*. POPL, 1997.
- [2] Freeman, T. and Pfenning, F. *Refinement types for ML*. PLDI, 1991.
- [3] Coalition for Content Provenance and Authenticity. *C2PA Technical Specification*. <https://c2pa.org>, 2024.
- [4] Zheng, L. et al. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS Datasets and Benchmarks, 2023.
- [5] Hintze, A., Proschinger Astrom, F., and Schossau, J. *Autonomous language-image generation loops converge to generic visual motifs*. Patterns (Cell Press), 2025. DOI: 10.1016/j.patter.2025.101451.
- [6] Nemecek, A., He, H., Cheng, G., and Ayday, E. *Authenticated Contradictions from Desynchronized Provenance and Watermarking*. arXiv:2603.02378, 2026.